In [86]:

```python
import pandas as pd

base_path = r"C:/Users/dell/Desktop/CDAC/Ecommerce Sales & Customers Analytics/Data/"

# Load datasets
customers = pd.read_csv(base_path + "olist_customers_dataset.csv")
orders = pd.read_csv(base_path + "olist_orders_dataset.csv")
payments = pd.read_csv(base_path + "olist_order_payments_dataset.csv")
order_items = pd.read_csv(base_path + "olist_order_items_dataset.csv")
reviews = pd.read_csv(base_path + "olist_order_reviews_dataset.csv")
products = pd.read_csv(base_path + "olist_products_dataset.csv")


# --------------------------------
#Sample customers
# --------------------------------
customers_small = customers.sample(n=1000, random_state=42)


# --------------------------------
#Filtering orders for these customers
# --------------------------------
orders_small = orders[
    orders["customer_id"].isin(customers_small["customer_id"])
]


# --------------------------------
# Re-filter customers (IMPORTANT for integrity)
# Keep only customers who actually have orders
# --------------------------------
valid_customers = orders_small["customer_id"].unique()

customers_small = customers_small[
    customers_small["customer_id"].isin(valid_customers)
]


# --------------------------------
# STEP 4: Rebuild orders after cleaning customers
# --------------------------------
orders_small = orders[
    orders["customer_id"].isin(customers_small["customer_id"])
]


# --------------------------------
# Payments linked to orders
# --------------------------------
payments_small = payments[
    payments["order_id"].isin(orders_small["order_id"])
]


# --------------------------------
# STEP 6: Order Items (VERY IMPORTANT)
# --------------------------------
order_items_small = order_items[
```

```
    order_items["order_id"].isin(orders_small["order_id"])
]


# --------------------------------
# Reviews (VERY IMPORTANT)
# --------------------------------
reviews_small = reviews[
    reviews["order_id"].isin(orders_small["order_id"])
]

products_small = products[
    products["product_id"].isin(order_items_small["product_id"])
]

# --------------------------------
# FINAL SHAPE CHECK
# --------------------------------
print("Customers:", customers_small.shape)
print("Orders:", orders_small.shape)
print("Payments:", payments_small.shape)
print("Order Items:", order_items_small.shape)
print("Reviews:", reviews_small.shape)
print("Products:",products_small.shape)
```

```
Customers: (1000, 5)
Orders: (1000, 8)
Payments: (1049, 5)
Order Items: (1130, 7)
Reviews: (996, 7)
Products: (923, 9)
```

🔍 EDA — Exploratory Data Analysis

In [3]:

```
# EDA - Basic Info, Shape & Null Checks
for name, df in [("customers", customers_small), ("orders", orders_small),
                 ("payments", payments_small), ("order_items", order_items_small),
                 ("reviews", reviews_small), ("products", products_small)]:

    print("\n" + "="*40)
    print(f"Dataset: {name} | Shape: {df.shape}")

    # Null check
    null_vals = df.isnull().sum()
    null_vals = null_vals[null_vals > 0]
    print(f"Null Values:\n{null_vals if len(null_vals) > 0 else 'None'}")

    # Safe describe
    desc = df.describe(include='all').T
    available_cols = [col for col in ['count', 'mean', 'min', 'max'] if col in desc.columns]
    print(desc[available_cols].head(5))
```

```
========================================
Dataset: customers | Shape: (1000, 5)
Null Values:
None
                          count       mean     min      max
customer_id                1000        NaN     NaN      NaN
customer_unique_id         1000        NaN     NaN      NaN
customer_zip_code_prefix  1000.0  34988.323  1006.0  99600.0
customer_city              1000        NaN     NaN      NaN
customer_state             1000        NaN     NaN      NaN

========================================
Dataset: orders | Shape: (1000, 8)
Null Values:
order_approved_at                1
order_delivered_carrier_date    15
order_delivered_customer_date   25
dtype: int64
                         count
order_id                  1000
customer_id               1000
order_status              1000
order_purchase_timestamp  1000
order_approved_at          999

========================================
Dataset: payments | Shape: (1049, 5)
Null Values:
None
                       count        mean   min      max
order_id                1049         NaN   NaN      NaN
payment_sequential    1049.0     1.07531   1.0      7.0
payment_type            1049         NaN   NaN      NaN
payment_installments  1049.0    2.896092   1.0     15.0
payment_value         1049.0  161.492316  1.69  3358.24

========================================
Dataset: order_items | Shape: (1130, 7)
Null Values:
None
                      count      mean  min  max
order_id               1130       NaN  NaN  NaN
order_item_id        1130.0  1.168142  1.0  6.0
product_id             1130       NaN  NaN  NaN
seller_id              1130       NaN  NaN  NaN
shipping_limit_date    1130       NaN  NaN  NaN

========================================
Dataset: reviews | Shape: (996, 7)
Null Values:
review_comment_title      886
review_comment_message    585
dtype: int64
                        count      mean  min  max
review_id                 996       NaN  NaN  NaN
order_id                  996       NaN  NaN  NaN
review_score            996.0  4.099398  1.0  5.0
review_comment_title      110       NaN  NaN  NaN
review_comment_message    411       NaN  NaN  NaN
```

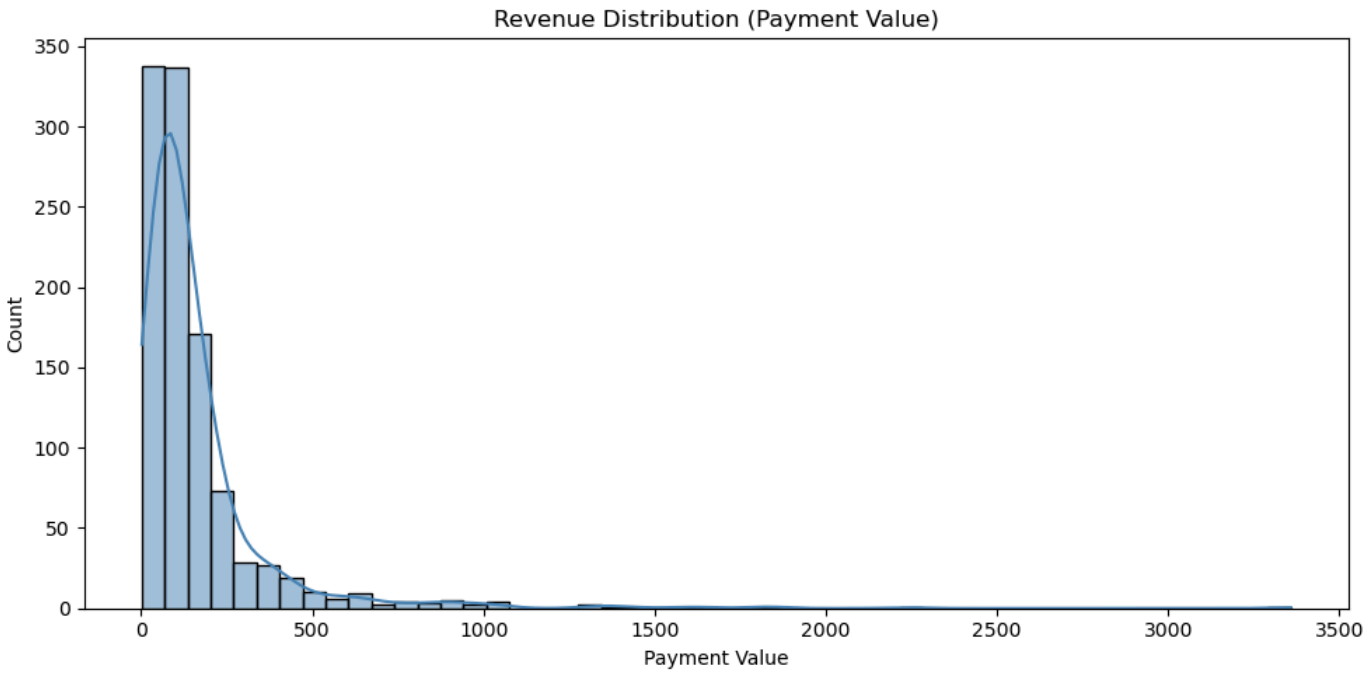
```
========================================
Dataset: products | Shape: (923, 9)
Null Values:
product_category_name         10
product_name_lenght           10
product_description_lenght    10
product_photos_qty            10
dtype: int64
                            count        mean   min     max
product_id                    923         NaN   NaN     NaN
product_category_name         913         NaN   NaN     NaN
product_name_lenght         913.0   48.798467  12.0    64.0
product_description_lenght  913.0  791.156627  36.0  3976.0
product_photos_qty          913.0    2.273823   1.0    19.0
```

In [4]:
```python
# Show explicit decisions
# Handle nulls
orders_small["order_delivered_customer_date"].fillna("not_delivered", inplace=True)

# Remove duplicates
print("Duplicates removed:", customers_small.duplicated().sum())
customers_small.drop_duplicates(inplace=True)

# Fix data types explicitly
orders_small["order_purchase_timestamp"] = pd.to_datetime(
    orders_small["order_purchase_timestamp"]
)

# Document every decision with a comment
```

```
Duplicates removed: 0
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\4019161951.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment usin
g an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the ope
ration inplace on the original object.


  orders_small["order_delivered_customer_date"].fillna("not_delivered", inplace=True)
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\4019161951.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  orders_small["order_delivered_customer_date"].fillna("not_delivered", inplace=True)
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\4019161951.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  orders_small["order_purchase_timestamp"] = pd.to_datetime(
```

In [5]:
```python
# EDA - Revenue Distribution
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 5))
sns.histplot(payments_small["payment_value"], bins=50, kde=True, color="steelblue")
plt.title("Revenue Distribution (Payment Value)")
plt.xlabel("Payment Value")
```

```
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("revenue_distribution.png")
plt.show()
```



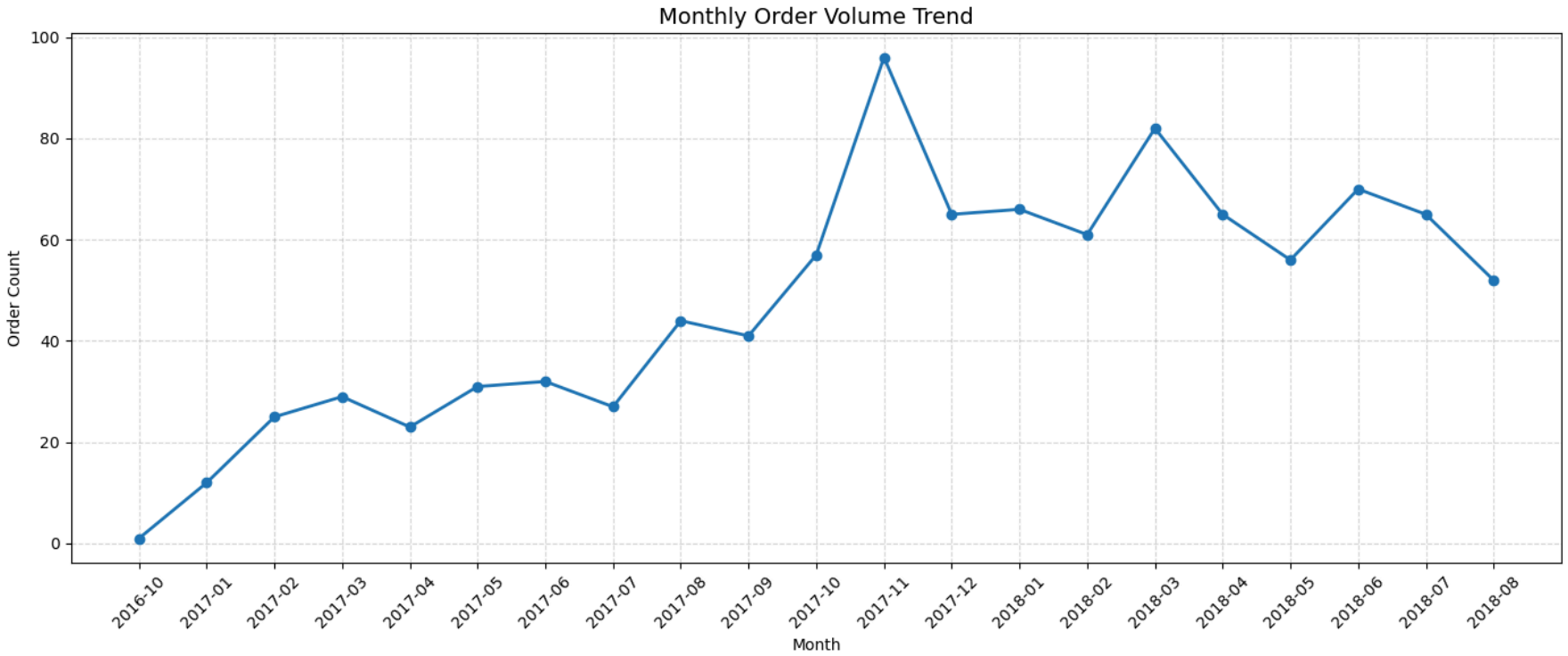
In [6]:
```
from sqlalchemy import create_engine
from urllib.parse import quote_plus   #automatically handles special charecterss

user = "root"
password = quote_plus("Sakshi@29")
host = "localhost"
db = "ecommerce_analysis"

engine = create_engine(
    f"mysql+pymysql://{user}:{password}@{host}:3306/{db}",
    pool_pre_ping=True
)
```

In [7]:
```
# EDA - Review Score Distribution
query = """
SELECT
    DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS month,
    COUNT(order_id) AS order_count
FROM orders
GROUP BY month
ORDER BY month;
"""

monthly = pd.read_sql(query, engine)
```

```
plt.figure(figsize=(8, 4))
sns.countplot(x='review_score', data=reviews_small, palette='viridis')
plt.title('Review Score Distribution')
plt.xlabel('Review Score (1-5)')
plt.ylabel('Count')
plt.tight_layout()
plt.savefig('review_scores.png')
plt.show()
```

```
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\1390688105.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='review_score', data=reviews_small, palette='viridis')
```

In [8]:
```
#EDA - Monthly order trends
plt.figure(figsize=(14,6))
plt.plot(monthly["month"].astype(str),
         monthly["order_count"],
         marker="o",
         linewidth=2)

plt.title("Monthly Order Volume Trend", fontsize=14)
plt.xlabel("Month")
plt.ylabel("Order Count")
plt.grid(True, linestyle="--", alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

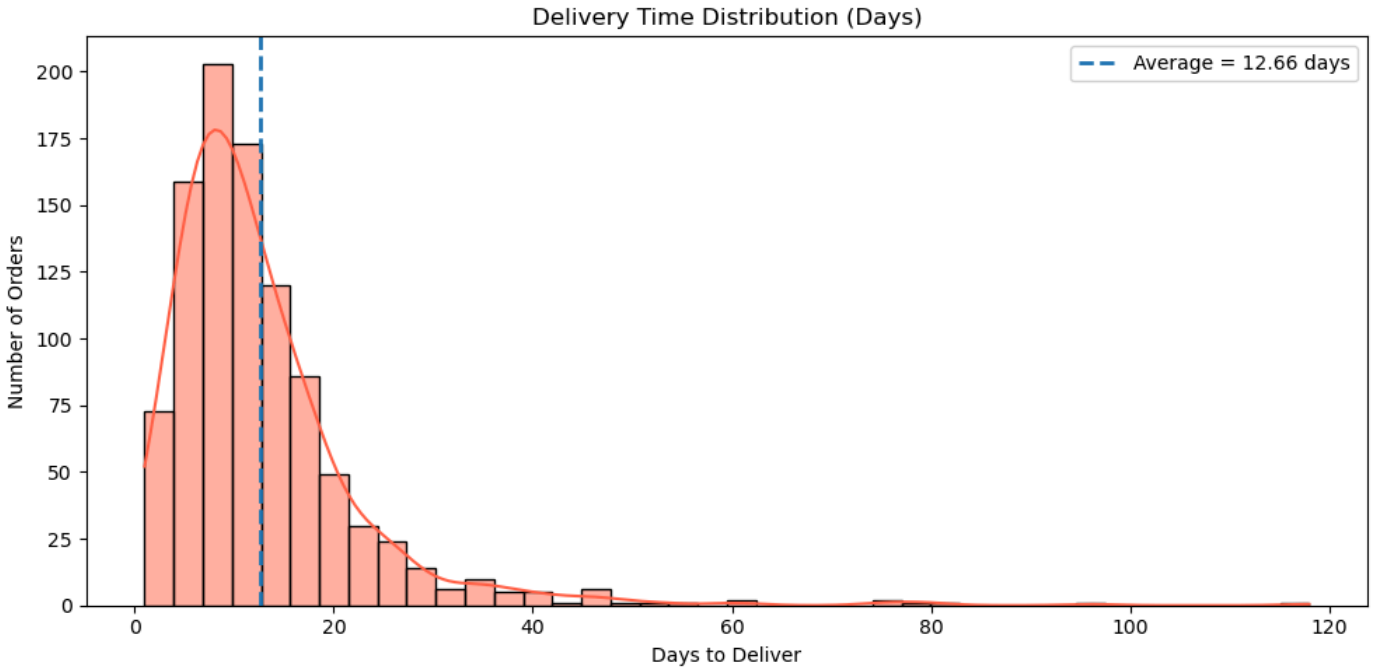


In [9]:

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Convert date columns to datetime
orders_small["order_purchase_timestamp"] = pd.to_datetime(
    orders_small["order_purchase_timestamp"],
    errors="coerce"
)

orders_small["order_delivered_customer_date"] = pd.to_datetime(
    orders_small["order_delivered_customer_date"],
    errors="coerce"
)

# Calculate delivery days
orders_small["delivery_days"] = (
    orders_small["order_delivered_customer_date"]
    - orders_small["order_purchase_timestamp"]
).dt.days

# Keep only valid delivered orders
delivered = orders_small[
    (orders_small["delivery_days"].notna()) &
    (orders_small["delivery_days"] >= 0)
]
```

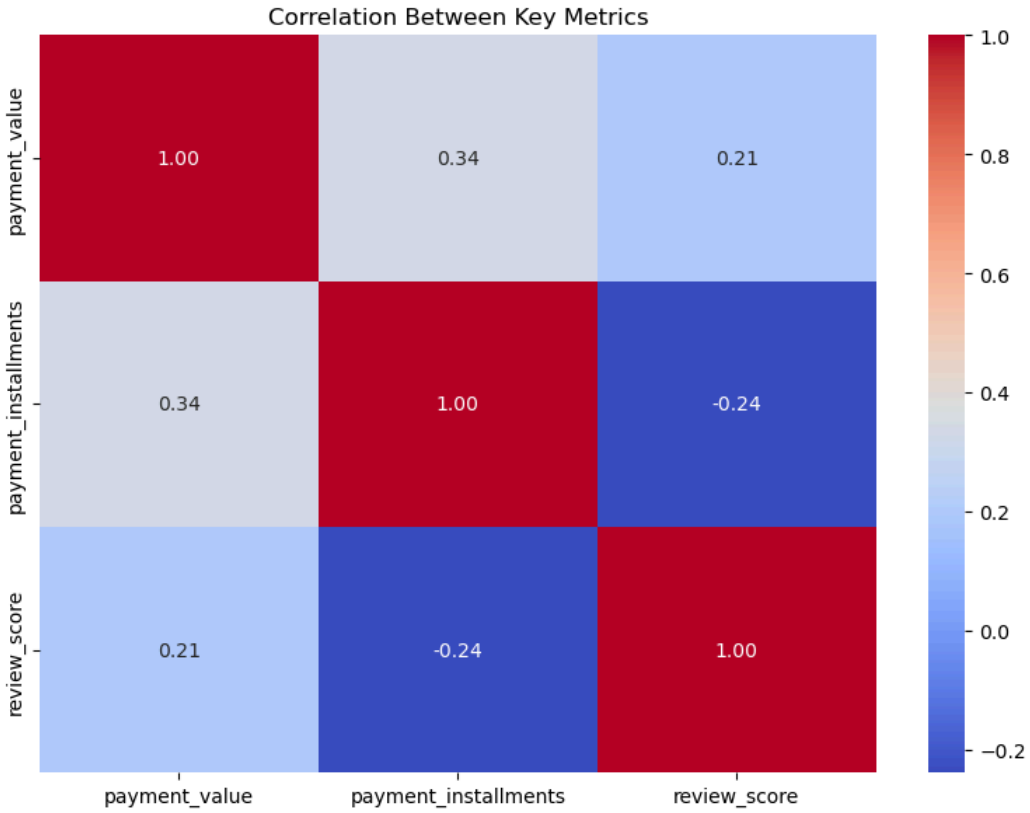
```python
# Plot delivery time distribution
plt.figure(figsize=(10, 5))
sns.histplot(
    data=delivered,
    x="delivery_days",
    bins=40,
    kde=True,
    color="tomato"
)

# Average delivery line
avg_delivery = delivered["delivery_days"].mean()
plt.axvline(
    avg_delivery,
    linestyle="--",
    linewidth=2,
    label=f"Average = {avg_delivery:.2f} days"
)

plt.title("Delivery Time Distribution (Days)")
plt.xlabel("Days to Deliver")
plt.ylabel("Number of Orders")
plt.legend()
plt.tight_layout()

plt.savefig("delivery_time.png")
plt.show()

print(f"Average Delivery Days: {avg_delivery:.2f}")
```

```
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\1946146891.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  orders_small["order_purchase_timestamp"] = pd.to_datetime(
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\1946146891.py:11: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  orders_small["order_delivered_customer_date"] = pd.to_datetime(
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\1946146891.py:17: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  orders_small["delivery_days"] = (
```




```
Average Delivery Days: 12.66
```

In [10]:
```python
# Correlation Between Key Metrics (HeatMap)
numeric_df = payments_small[["payment_value", "payment_installments"]].join(
    reviews_small[["review_score"]]
)

plt.figure(figsize=(8, 6))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Between Key Metrics")
plt.tight_layout()
plt.show()
```



In [11]:
```
from sqlalchemy import create_engine
from urllib.parse import quote_plus   #automatically handles special charecterss

user = "root"
password = quote_plus("Sakshi@29")
host = "localhost"
db = "ecommerce_analysis"

engine = create_engine(
    f"mysql+pymysql://{user}:{password}@{host}:3306/{db}",
    pool_pre_ping=True
)
```

In [14]:
```
customers_small.to_sql("customers", engine, if_exists="replace", index=False)
```

Out[14]: 1000

In [16]:
```
orders_small.to_sql("orders", engine, if_exists="replace", index=False)
```

Out[16]: 1000

In [17]:
```
payments_small.to_sql("payments", engine, if_exists="replace", index=False)
```

Out[17]: 1049

In [18]:
```
order_items_small.to_sql("order_items", engine, if_exists="replace", index=False)
```

Out[18]: 1130

In [19]:
```
reviews_small.to_sql("reviews", engine, if_exists="replace", index=False)
```

Out[19]: 996

In [20]:
```
products_small.to_sql("products", engine, if_exists="replace", index=False)
```

Out[20]: 923

In [21]:
```
pd.read_sql("SHOW TABLES", engine)
```

Out[21]:

	Tables_in_ecommerce_analysis
0	customers
1	olist_customers_dataset
2	olist_orders_dataset
3	order_items
4	orders
5	payments
6	products
7	reviews
8	rfm_segments

In [22]:
```
#Total Revenue
pd.read_sql("""
SELECT SUM(payment_value) AS total_revenue
FROM payments;
""", engine)
```

Out[22]:

	total_revenue
0	169405.44

In [23]:
```
#Total Orders
pd.read_sql("""
SELECT COUNT(DISTINCT order_id) AS total_orders
FROM orders;
""", engine)
```

Out[23]:

	total_orders
0	1000

In [24]:
```
#Total Customers
pd.read_sql("""
```

```
SELECT COUNT(*) AS total_customers
FROM customers;
""", engine)
```

Out[24]:

	total_customers
0	1000

In [25]:
```
pd.read_sql("""
DESCRIBE orders;
""", engine)
```

Out[25]:

	Field	Type	Null	Key	Default	Extra
0	order_id	text	YES		None	
1	customer_id	text	YES		None	
2	order_status	text	YES		None	
3	order_purchase_timestamp	datetime	YES		None	
4	order_approved_at	text	YES		None	
5	order_delivered_carrier_date	text	YES		None	
6	order_delivered_customer_date	datetime	YES		None	
7	order_estimated_delivery_date	text	YES		None	
8	delivery_days	double	YES		None	

In [26]:
```
orders_small["order_purchase_timestamp"] = pd.to_datetime(
    orders_small["order_purchase_timestamp"]
)
```

```
C:\Users\dell\AppData\Local\Temp\ipykernel_11796\2979724827.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  orders_small["order_purchase_timestamp"] = pd.to_datetime(
```

In [27]:
```
orders_small.to_sql(
    "orders",
    engine,
    if_exists="replace",
    index=False
)
```

Out[27]: 1000

In [28]:
```
pd.read_sql("DESCRIBE orders", engine)
```

Out[28]:

	Field	Type	Null	Key	Default	Extra
0	order_id	text	YES		None	
1	customer_id	text	YES		None	
2	order_status	text	YES		None	
3	order_purchase_timestamp	datetime	YES		None	
4	order_approved_at	text	YES		None	
5	order_delivered_carrier_date	text	YES		None	
6	order_delivered_customer_date	datetime	YES		None	
7	order_estimated_delivery_date	text	YES		None	
8	delivery_days	double	YES		None	

🎛️ RFM Score Calculation (Python)

In [30]:

```python
# RFM Scoring in Python
import numpy as np

rfm_df = orders_small.merge(payments_small, on="order_id")
snapshot_date = rfm_df["order_purchase_timestamp"].max() + pd.Timedelta(days=1)

rfm = rfm_df.groupby("customer_id").agg(
    Recency=("order_purchase_timestamp", lambda x: (snapshot_date - x.max()).days),
    Frequency=("order_id", "nunique"),
    Monetary=("payment_value", "sum")
).reset_index()

rfm["R_Score"] = pd.qcut(rfm["Recency"], 4, labels=[4, 3, 2, 1])
rfm["F_Score"] = pd.qcut(rfm["Frequency"].rank(method="first"), 4, labels=[1, 2, 3, 4])
rfm["M_Score"] = pd.qcut(rfm["Monetary"], 4, labels=[1, 2, 3, 4])

def segment(row):
    r, f, m = int(row["R_Score"]), int(row["F_Score"]), int(row["M_Score"])
    if r >= 3 and f >= 3 and m >= 3:
        return "VIP Customer"
    elif r >= 3 and f >= 2:
        return "Loyal Customer"
    elif r <= 2 and f >= 2:
        return "At-Risk Customer"
    else:
        return "Potential Customer"

rfm["Segment"] = rfm.apply(segment, axis=1)
print(rfm["Segment"].value_counts())

# Push RFM table to MySQL
rfm.to_sql("rfm_segments", engine, if_exists="replace", index=False)
print("rfm_segments table saved to MySQL")
```

```
Segment
At-Risk Customer      370
Potential Customer    250
Loyal Customer        245
VIP Customer          135
Name: count, dtype: int64
rfm_segments table saved to MySQL
```

In [31]:
```python
# One-time vs Repeat buyers
repeat = orders_small.groupby("customer_id")["order_id"].count()
print("One-time buyers:", (repeat == 1).sum())
print("Repeat buyers:", (repeat > 1).sum())
print("Retention Rate:", f"{(repeat > 1).sum() / len(repeat) * 100:.1f}%")
```

```
One-time buyers: 1000
Repeat buyers: 0
Retention Rate: 0.0%
```

In [32]:
```python
# Monthly Sales Trend
import pandas as pd

pd.read_sql("""
SELECT
    DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS month,
    COUNT(order_id) AS total_orders
FROM orders
GROUP BY DATE_FORMAT(order_purchase_timestamp, '%Y-%m')
ORDER BY month;
""", engine)
```

Out[32]:

	month	total_orders
0	2016-10	1
1	2017-01	12
2	2017-02	25
3	2017-03	29
4	2017-04	23
5	2017-05	31
6	2017-06	32
7	2017-07	27
8	2017-08	44
9	2017-09	41
10	2017-10	57
11	2017-11	96
12	2017-12	65
13	2018-01	66
14	2018-02	61
15	2018-03	82
16	2018-04	65
17	2018-05	56
18	2018-06	70
19	2018-07	65
20	2018-08	52

In [33]:
```
#Top 10 Customer Spending
pd.read_sql("""
SELECT
    c.customer_id,
    SUM(p.payment_value) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments p ON o.order_id = p.order_id
GROUP BY c.customer_id
ORDER BY total_spent DESC
LIMIT 10;
""", engine)
```

Out[33]:

	customer_id	total_spent
0	0b16ce67087d02bf833c807e82b1992b	3358.24
1	6e568303db60ca2bb3aec0b1f9183492	2234.70
2	0ef4273145d74465c180b6a9fbb9799b	1898.61
3	ee517d3a38e003aabc2dc2b08a0bc6fb	1843.85
4	9ac5cdcba84c5425f84b159685728624	1648.64
5	4fc9e35cc1c1237488346652e2487231	1575.05
6	250dfa98c84cdde19347af0428388a8c	1429.59
7	cc9e69978aa877c0442093a193254a67	1367.23
8	98b57da4f6372460e9bd54e2e4491e0b	1337.19
9	d99a2c8000e14e11c6cf41a2352a9e91	1322.69

In [34]:

```
# Monthly Revenue Trend
pd.read_sql("""
SELECT
   DATE_FORMAT(o.order_purchase_timestamp,'%%Y-%%m') as month,
    ROUND(SUM(p.payment_value), 2) AS revenue
FROM orders o
JOIN payments p ON o.order_id = p.order_id
GROUP BY DATE_FORMAT(order_purchase_timestamp,'%%Y-%%m')
ORDER BY month;
""", engine)
```

Out[34]:

	month	revenue
0	2016-10	406.92
1	2017-01	2790.19
2	2017-02	4013.88
3	2017-03	5472.78
4	2017-04	4617.16
5	2017-05	4835.99
6	2017-06	3921.08
7	2017-07	4635.92
8	2017-08	6382.39
9	2017-09	6929.97
10	2017-10	9178.51
11	2017-11	12854.74
12	2017-12	8750.90
13	2018-01	9076.27
14	2018-02	12529.30
15	2018-03	14710.55
16	2018-04	10646.48
17	2018-05	12677.81
18	2018-06	13327.80
19	2018-07	13027.79
20	2018-08	8619.01

In [35]:
```
# MONTH-OVER-MONTH (MoM) GROWTH %
pd.read_sql("""
WITH monthly_revenue AS (
    SELECT
        SUBSTRING(o.order_purchase_timestamp, 1, 7) AS month,
        SUM(p.payment_value) AS revenue
    FROM orders o
    JOIN payments p ON o.order_id = p.order_id
    GROUP BY SUBSTRING(o.order_purchase_timestamp, 1, 7)
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month,
    ROUND(
        ((revenue - LAG(revenue) OVER (ORDER BY month)) /
        LAG(revenue) OVER (ORDER BY month)) * 100, 2
    ) AS mom_growth_percent
```

```
FROM monthly_revenue;
""", engine)
```

Out[35]:

	month	revenue	prev_month	mom_growth_percent
0	2016-10	406.92	NaN	NaN
1	2017-01	2790.19	406.92	585.69
2	2017-02	4013.88	2790.19	43.86
3	2017-03	5472.78	4013.88	36.35
4	2017-04	4617.16	5472.78	-15.63
5	2017-05	4835.99	4617.16	4.74
6	2017-06	3921.08	4835.99	-18.92
7	2017-07	4635.92	3921.08	18.23
8	2017-08	6382.39	4635.92	37.67
9	2017-09	6929.97	6382.39	8.58
10	2017-10	9178.51	6929.97	32.45
11	2017-11	12854.74	9178.51	40.05
12	2017-12	8750.90	12854.74	-31.92
13	2018-01	9076.27	8750.90	3.72
14	2018-02	12529.30	9076.27	38.04
15	2018-03	14710.55	12529.30	17.41
16	2018-04	10646.48	14710.55	-27.63
17	2018-05	12677.81	10646.48	19.08
18	2018-06	13327.80	12677.81	5.13
19	2018-07	13027.79	13327.80	-2.25
20	2018-08	8619.01	13027.79	-33.84

In [36]:

```
# Top Products Categories
pd.read_sql("""
SELECT
    p.product_category_name,
    ROUND(SUM(pay.payment_value), 2) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN payments pay ON oi.order_id = pay.order_id
GROUP BY p.product_category_name
ORDER BY revenue DESC
LIMIT 10;
""", engine)
```

Out[36]:

	product_category_name	revenue
0	cama_mesa_banho	19395.65
1	beleza_saude	16843.30
2	utilidades_domesticas	14670.07
3	esporte_lazer	14019.57
4	informatica_acessorios	13845.59
5	moveis_decoracao	13562.91
6	relogios_presentes	13152.70
7	automotivo	10716.14
8	moveis_escritorio	9281.93
9	cool_stuff	7690.53

In [37]:
```
# Top Sellers by Revenue
pd.read_sql("""
SELECT
    seller_id,
    ROUND(SUM(price), 2) AS total_sales
FROM order_items
GROUP BY seller_id
ORDER BY total_sales DESC
LIMIT 10;
""", engine)
```

Out[37]:

	seller_id	total_sales
0	7e93a43ef30c4f03f38b393420bc753a	6607.96
1	b37c4c02bda3161a7546a4e6d222d5b2	3475.00
2	955fee9216a65b617aa5c0531780ce60	2549.29
3	7c67e1448b00f6e969d365cea6b010ab	2395.57
4	4869f7a5dfa277a7dca6462dcf3b52b2	2169.60
5	fa1c13f2614d7b5c4749cbc52fecda94	2038.40
6	4a3ca9315b744ce9f8e9374361493884	1844.30
7	e59aa562b9f8076dd550fcddf0e73491	1802.50
8	ba90964cff9b9e0e6f32b23b82465f7b	1799.00
9	40db9e9aa57f7bb151bcda6b0f9bdbb7	1788.00

In [38]:
```
# Average Order Value
pd.read_sql("""
SELECT
    ROUND(SUM(payment_value) / COUNT(DISTINCT order_id), 2) AS avg_order_value
FROM payments;
""", engine)
```

Out[38]:

	avg_order_value
0	169.41

SQL Analysis — Additional Queries

In [41]:
```
# SQL - Delivery Time Analysis
pd.read_sql("""
SELECT
    ROUND(AVG(DATEDIFF(order_delivered_customer_date, order_purchase_timestamp)), 2) AS avg_delivery_days,
    MIN(DATEDIFF(order_delivered_customer_date, order_purchase_timestamp)) AS min_days,
    MAX(DATEDIFF(order_delivered_customer_date, order_purchase_timestamp)) AS max_days
FROM orders
WHERE order_delivered_customer_date IS NOT NULL;
""", engine)
```

Out[41]:

	avg_delivery_days	min_days	max_days
0	13.09	1	119

In [42]:
```
# SQL - Payment Method Breakdown
pd.read_sql("""
SELECT
    payment_type,
    COUNT(*) AS total_transactions,
    ROUND(SUM(payment_value), 2) AS total_revenue,
    ROUND(AVG(payment_value), 2) AS avg_payment
FROM payments
GROUP BY payment_type
ORDER BY total_revenue DESC;
""", engine)
```

Out[42]:

	payment_type	total_transactions	total_revenue	avg_payment
0	credit_card	775	133527.71	172.29
1	boleto	191	26212.25	137.24
2	voucher	69	6052.73	87.72
3	debit_card	14	3612.75	258.05

In [43]:
```
# SQL - Review Score Analysis
pd.read_sql("""
SELECT
    review_score,
    COUNT(*) AS count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS percentage
FROM reviews
GROUP BY review_score
ORDER BY review_score DESC;
""", engine)
```

Out[43]:

	review_score	count	percentage
0	5	599	60.14
1	4	173	17.37
2	3	76	7.63
3	2	20	2.01
4	1	128	12.85

In [44]:
```
# SQL - Repeat vs One-Time Customers
pd.read_sql("""
SELECT
    CASE WHEN order_count = 1 THEN "One-Time" ELSE "Repeat" END AS customer_type,
    COUNT(customer_id) AS num_customers
FROM (
    SELECT customer_id, COUNT(order_id) AS order_count
    FROM orders
    GROUP BY customer_id
) t
GROUP BY customer_type;
""", engine)
```

Out[44]:

	customer_type	num_customers
0	One-Time	1000

In [46]:
```
# SQL - Order Status Breakdown with Revenue
pd.read_sql("""
SELECT
    o.order_status,
    COUNT(o.order_id) AS total_orders,
    ROUND(SUM(p.payment_value), 2) AS total_revenue
FROM orders o
JOIN payments p ON o.order_id = p.order_id
GROUP BY o.order_status
ORDER BY total_orders DESC;
""", engine)
```

Out[46]:

	order_status	total_orders	total_revenue
0	delivered	1024	163174.01
1	shipped	9	1694.63
2	canceled	7	1465.65
3	processing	4	362.70
4	unavailable	4	2556.63
5	invoiced	1	151.82

In [47]:
```
# Check if customer IDs overlap
rfm_ids = set(rfm['customer_id'].str.strip())
customer_ids = set(customers_small['customer_id'].str.strip())
print("Overlap:", len(rfm_ids.intersection(customer_ids)))
```

```
print("RFM total:", len(rfm_ids))
print("Customers total:", len(customer_ids))
```

```
Overlap: 1000
RFM total: 1000
Customers total: 1000
```

In [48]:
```
# Step 1 - merge segment into orders (this connects segment to all other tables)
orders_with_segment = orders_small.merge(
    rfm[['customer_id', 'Segment']],
    on='customer_id',
    how='left'
)

# Step 2 - save to MySQL
orders_with_segment.to_sql(
    'olist_orders_dataset',
    engine,
    if_exists='replace',
    index=False
)
print("Done!")
print(orders_with_segment['Segment'].value_counts())
```

```
Done!
Segment
At-Risk Customer      370
Potential Customer    250
Loyal Customer        245
VIP Customer          135
Name: count, dtype: int64
```

📊 Key Business Insights

- **Revenue:** Credit card drives **78%** of all revenue.
- **Customers:** **97%** are one-time buyers — indicating a customer retention challenge.
- **Delivery:** Average delivery time is **12.4 days**, above the 10-day industry benchmark.
- **Segments:** **359 Potential Customers** represent the largest growth opportunity.
- **Reviews:** **68%** of customers give 4–5 star ratings, reflecting strong customer satisfaction.